# Package 'LatencyA'

October 21, 2026

**Type** Package

**Title** Latency Analysis in Epidemiological Studies Using Weighted Cumulative Exposures

**Version** 1.0.2

**Maintainer** Fuyu Guo <fuyuguo@hsph.harvard.edu>

**Description** Implements weighted cumulative exposure Cox proportional hazards models for latency analysis with repeatedly measured, time-varying exposures. Instead of entering every lagged exposure as a separate and highly correlated covariate, the lag-specific log hazard ratios are constrained to follow a smooth function of the time since exposure, parameterized by natural cubic splines or polynomials, so that the susceptible window of an exposure can be identified from the estimated lag-risk curve. Functions are provided to fit models with a prespecified number of knots or polynomial degree, to select the number of knots by a grid search or the spline terms by a stepwise search using the Akaike or Bayesian information criterion, and to extract lag-specific log hazard ratios with their variances. For the data-driven model selection procedures, inference is based on a cluster bootstrap that resamples subjects, which can be run sequentially or in parallel. The methods are described in Guo, Qian, Wang, Wang, Rosner and Wang (2026) ``Taking Into Account the Susceptible Window for Exposures in Cox Proportional Hazards Model: A Way Using Splines to Model the Lag-Risk Relationship" <doi:10.1002/sim.70755>.

**Imports** Hmisc, survival, stats, MASS, stringr, future, furrr

**License** MIT + file LICENSE

**Encoding** UTF-8

**LazyData** true

**Depends** R (>= 4.1.0)

**Config/roxygen2/version** 8.0.0

**NeedsCompilation** no

**Author** Fuyu Guo [aut, cre],
Zihan Qian [aut],
Molin Wang [aut]

**Repository** CRAN

**Date/Publication** 2026-10-21 14:10:02 UTC

# Contents

CoxKnotsearch . . . . . 2
CoxKnotsearch_boot . . . . . 3
CoxNCSpline . . . . . 5
CoxPoly . . . . . 6
CoxTermsearch . . . . . 7
CoxTermsearch_boot . . . . . 9
extract_CoxKnotsearch . . . . . 10
extract_CoxNCSpline . . . . . 11
extract_CoxPoly . . . . . 12
extract_CoxTermsearch . . . . . 13
resample_clusters . . . . . 14
sim_data . . . . . 14

**Index** **15**

---

`CoxKnotsearch` *This function implments GridSearch of natural cubic splines in Cox regression for latency analysis This is GreadSearching by knots*

---

## Description

This function implments GridSearch of natural cubic splines in Cox regression for latency analysis This is GreadSearching by knots

## Usage

```
CoxKnotsearch(
  data,
  time_start,
  time_end,
  status,
  exposure,
  knots_number,
  latency,
  adjusted_variable = NULL,
  adjusted_model = NULL,
  criteria = "AIC"
)
```

### Arguments

| | |
|---|---|
| data | A data frame containing the data |
| time_start | start time |
| time_end | end time |
| status | survival status. 1 for death and 0 for censored |
| exposure | Name of the time-varying exposure variable. From the most recent to the furthest in time. |
| knots_number | A vector of potential number of knots |
| latency | pre-specified latency |
| adjusted_variable | A vector of adjusted variables |
| adjusted_model | A vector of adjusted models |
| criteria | A string to specify the criteria for model selection. It can be either "AIC" or "BIC". The default is "AIC" |

### Value

A list. The first element is the best knots with the lowest AIC. The second element is a Cox model

### Examples

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:19)
# knots can be set as as a vector of potential number of knots, here we use 3 to save time
knots_number <- c(3)
latency <- 20
fit <- CoxKnotsearch(sim_data, time_start, time_end, status, exposure, knots_number, latency)
```

---

## CoxKnotsearch_boot *This function conduct bootstraps for @CoxKnotsearch*

---

### Description

This function conduct bootstraps for @CoxKnotsearch

### Usage

```
CoxKnotsearch_boot(
  data,
  time_start,
  time_end,
  status,
  exposure,
```

```
  knots_number,
  latency,
  adjusted_variable = NULL,
  adjusted_model = NULL,
  lag,
  parallel = FALSE,
  parallel_plan = c("multicore", "multisession"),
  boot_iter = 10,
  id = "id",
  criteria = "AIC"
)
```

## Arguments

| | |
|---|---|
| data | A data frame containing the data |
| time_start | start time |
| time_end | end time |
| status | survival status. 1 for death and 0 for censored |
| exposure | Name of the time-varying exposure variable. From the most recent to the furthest in time. |
| knots_number | A vector of prespecified knot number |
| latency | prespecified latency |
| adjusted_variable | A vector of adjusted variables |
| adjusted_model | A vector of adjusted models |
| lag | A value of lag time |
| parallel | A boolean numeric to indicate whether to run the function in parallel |
| parallel_plan | A string to specify the future plan used when parallel = TRUE. It can be either "multicore" or "multisession". The default is "multicore" |
| boot_iter | Number of bootstrap iterations |
| id | A string of the column name that contains the subject ID |
| criteria | A string to specify the criteria for model selection. It can be either "AIC" or "BIC". The default is "AIC" |

## Value

A data.frame. The first column is the mean of log HR. The second column is the variance of log HR, the third and fourth columns are the 2.5 and 97.5 percentiles of log HR if parallel = TRUE

## Examples

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:15)
```

```
# knots can be set as as a vector of potential number of knots, here we use 3 to save time
knots_number <- c(3)
latency <- 16
adjusted_variable <- "L"
adjusted_model <- "L"
lag <- 2
boot_iter <- 2
id <- "id"
# use a subset of subjects to keep the example fast
ids <- unique(sim_data$id)[1:300]
small_data <- sim_data[sim_data$id %in% ids, ]
result <- CoxKnotsearch_boot(small_data, time_start, time_end, status,
exposure, knots_number = knots_number, latency,
lag = lag, parallel = FALSE, boot_iter = boot_iter, id = id)

# the bootstrap can also be run in parallel
result <- CoxKnotsearch_boot(small_data, time_start, time_end, status,
exposure, knots_number = knots_number, latency,
lag = lag, parallel = TRUE, boot_iter = boot_iter, id = id)
```

---

`CoxNCSpline` *This function implments natural cubic splines in Cox regression for latency analysis*

---

## Description

This function implments natural cubic splines in Cox regression for latency analysis

## Usage

```
CoxNCSpline(
  data,
  time_start,
  time_end,
  status,
  exposure,
  knots_number,
  latency,
  adjusted_variable = NULL,
  adjusted_model = NULL,
  id = "id"
)
```

## Arguments

| | |
|---|---|
| `data` | A data frame containing the data |
| `time_start` | start time |

| | |
|---|---|
| time_end | end time |
| status | survival status. 1 for death and 0 for censored |
| exposure | Name of the time-varying exposure variable. From the most recent to the furthest in time. |
| knots_number | Pre-specified number of knots |
| latency | prespecified latency |
| adjusted_variable | A vector of adjusted variables |
| adjusted_model | A vector of adjusted models |
| id | A string to specify the name of the id column |

## Value

A list. The first element is the best knots with the lowest AIC. The second element is a Cox model

## Examples

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:19)
knots_number <- 3
latency <- 20
fit <- CoxNCSpline(sim_data, time_start, time_end, status, exposure, knots_number, latency)
```

---

# CoxPoly *This is a function to implement polynomial Cox regression for latency analysis*

---

## Description

This is a function to implement polynomial Cox regression for latency analysis

## Usage

```
CoxPoly(
  data,
  time_start,
  time_end,
  status,
  exposure,
  degree,
  latency,
  adjusted_variable = NULL,
  adjusted_model = NULL,
  id = "id"
)
```

## Arguments

| | |
|---|---|
| data | A data frame containing the data |
| time_start | start time |
| time_end | end time |
| status | survival status. 1 for death and 0 for censored |
| exposure | Name of the time-varying exposure variable. From the most recent to the furthest in time. |
| degree | degree of the polynomial |
| latency | prespecified latency |
| adjusted_variable | A vector of adjusted variables |
| adjusted_model | A vector of adjusted models |
| id | A string to specify the name of the id column |

## Value

A Cox model object

## Examples

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:19)
degree <- 3
latency <- 20
fit <- CoxPoly(sim_data, time_start, time_end, status, exposure, degree, latency)
fit
```

---

# CoxTermsearch *This function implments Stepwise model selection by terms of natural cubic splines in Cox regression for latency analysis This is Gread-Searching by knots*

---

## Description

This function implments Stepwise model selection by terms of natural cubic splines in Cox regression for latency analysis This is GreadSearching by knots

## Usage

```
CoxTermsearch(
  data,
  time_start,
  time_end,
  status,
  exposure,
  knots,
  latency,
  adjusted_variable = NULL,
  adjusted_model = NULL,
  criteria = "AIC"
)
```

## Arguments

| | |
|---|---|
| data | A data frame containing the data |
| time_start | start time |
| time_end | end time |
| status | survival status. 1 for death and 0 for censored |
| exposure | Name of the time-varying exposure variable. From the most recent to the furthest in time. |
| knots | A vector of prespecified knots |
| latency | prespecified latency |
| adjusted_variable | A vector of adjusted variables |
| adjusted_model | A vector of adjusted models |
| criteria | A string to specify the criteria for model selection. It can be either "AIC" or "BIC". The default is "AIC" |

## Value

A list. The first element is the best knots with the lowest AIC. The second element is a Cox model

## Examples

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:15)
knots <- seq(0, 15, by = 3)
latency <- 16
adjusted_variable <- "L"
adjusted_model <- "L"
fit <- CoxTermsearch(sim_data, time_start, time_end, status, exposure, knots = knots, latency)
#In this example, we will generate a random covariate L and adjust for it in our model
sim_data$L <- rbinom(nrow(sim_data), 1, 0.5)
```

```
adjusted_variable <- "L"
adjusted_model <- "L+I(L^2)"
fit <- CoxTermsearch(sim_data, time_start, time_end, status, exposure, knots = knots, latency,
adjusted_variable = adjusted_variable, adjusted_model = adjusted_model)
#L is selected into the final model but L^2 was not
```

---

`CoxTermsearch_boot` *This function conduct bootstraps for @CoxTermsearch*

---

## Description

This function conduct bootstraps for @CoxTermsearch

## Usage

```
CoxTermsearch_boot(
  data,
  time_start,
  time_end,
  status,
  exposure,
  knots,
  latency,
  adjusted_variable = NULL,
  adjusted_model = NULL,
  lag,
  parallel = FALSE,
  parallel_plan = c("multicore", "multisession"),
  boot_iter = 10,
  id = "id",
  criteria = "AIC"
)
```

## Arguments

| | |
|---|---|
| `data` | A data frame containing the data |
| `time_start` | start time |
| `time_end` | end time |
| `status` | survival status. 1 for death and 0 for censored |
| `exposure` | Name of the time-varying exposure variable. From the most recent to the furthest in time. |
| `knots` | A vector of prespecified knots |
| `latency` | prespecified latency |
| `adjusted_variable` | A vector of adjusted variables |

| | |
|---|---|
| adjusted_model | A vector of adjusted models |
| lag | A value of lag time |
| parallel | A boolean numeric to indicate whether to run the function in parallel |
| parallel_plan | A string to specify the future plan used when parallel = TRUE. It can be either "multicore" or "multisession". The default is "multicore" |
| boot_iter | Number of bootstrap iterations |
| id | A string of the column name that contains the subject ID |
| criteria | A string to specify the criteria for model selection. It can be either "AIC" or "BIC". The default is "AIC" |

## Value

A data.frame. The first column is the mean of log HR. The second column is the variance of log HR, the third and fourth columns are the 2.5 and 97.5 percentiles of log HR if parallel = TRUE

## Examples

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:15)
knots <- seq(0, 15, by = 3)
latency <- 16
adjusted_variable <- "L"
adjusted_model <- "L"
lag <- 2
boot_iter <- 2
id <- "id"
# use a subset of subjects to keep the example fast
ids <- unique(sim_data$id)[1:300]
small_data <- sim_data[sim_data$id %in% ids, ]
result <- CoxTermsearch_boot(small_data, time_start, time_end,
status, exposure, knots = knots, latency,
lag = lag, parallel = FALSE, boot_iter = boot_iter, id = id)

# the bootstrap can also be run in parallel
result <- CoxTermsearch_boot(small_data, time_start, time_end,
status, exposure, knots = knots, latency,
lag = lag, parallel = TRUE, boot_iter = boot_iter, id = id)
```

---

# extract_CoxKnotsearch *This is a function to extract log HR based on the resuls from CoxKnotsearch*

---

## Description

This is a function to extract log HR based on the resuls from CoxKnotsearch

### Usage

```
extract_CoxKnotsearch(fit, lag, latency)
```

### Arguments

| | |
|---|---|
| fit | A model from CoxKnotsearch |
| lag | A lag time, or a vector of lag times |
| latency | prespecified latency |

### Value

A data frame with one row per lag, giving the estimated log HR at each lag time

### Examples

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:19)
# knots can be set as as a vector of potential number of knots, here we use 3 to save time
knots_number <- c(3)
latency <- 20
fit <- CoxKnotsearch(sim_data, time_start, time_end, status, exposure, knots_number, latency)
extract_CoxKnotsearch(fit, 10, latency = latency)
extract_CoxKnotsearch(fit, c(0, 5, 10), latency = latency)
```

---

## extract_CoxNCSpline *This is a function to extract log HR based on the resuls from CoxNCSpline*

---

### Description

This is a function to extract log HR based on the resuls from CoxNCSpline

### Usage

```
extract_CoxNCSpline(fit, lag, latency)
```

### Arguments

| | |
|---|---|
| fit | A model from CoxNCSpline |
| lag | A lag time, or a vector of lag times |
| latency | prespecified latency |

### Value

A data frame with one row per lag, giving the estimated log HR and its variance at each lag time

## Examples

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:19)
knots_number <- 3
latency <- 20
fit <- CoxNCSpline(sim_data, time_start, time_end, status, exposure, knots_number, latency)
extract_CoxNCSpline(fit, 10, latency = latency)
extract_CoxNCSpline(fit, c(0, 5, 10), latency = latency)
```

---

`extract_CoxPoly` *This is a function to extract log HR based on the resuls from CoxPoly*

---

## Description

This is a function to extract log HR based on the resuls from CoxPoly

## Usage

```
extract_CoxPoly(fit, lag)
```

## Arguments

| | |
|---|---|
| `fit` | A model from CoxPoly |
| `lag` | A lag time, or a vector of lag times |

## Value

A data frame with one row per lag, giving the estimated log HR and its variance at each lag time

## Examples

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:19)
degree <- 3
latency <- 20
fit <- CoxPoly(sim_data, time_start, time_end, status, exposure, degree, latency)
extract_CoxPoly(fit, 10)
extract_CoxPoly(fit, c(0, 5, 10))
```

`extract_CoxTermsearch` *This is a function to extract log HR based on the resuls from CoxTermsearch*

## Description

This is a function to extract log HR based on the resuls from CoxTermsearch

## Usage

```
extract_CoxTermsearch(fit, lag, latency, knots)
```

## Arguments

| | |
|---|---|
| `fit` | A model from CoxKnotsearch |
| `lag` | A lag time, or a vector of lag times |
| `latency` | prespecified latency |
| `knots` | A vector of prespecified knots |

## Value

A data frame with one row per lag, giving the estimated log HR at each lag time

## Examples

```
time_start <- "age_start"
time_end <- "age_end"
status <- "failure"
exposure <- paste0("lag", 0:19)
knots <- seq(0, 15, by = 3)
latency <- 20
fit <- CoxTermsearch(sim_data, time_start, time_end, status, exposure, knots = knots, latency)
extract_CoxTermsearch(fit, 10, latency = latency, knot = knots)
extract_CoxTermsearch(fit, c(0, 5, 10), latency = latency, knot = knots)
#In this example, we will generate a random covariate L and adjust for it in our model
sim_data$L <- rbinom(nrow(sim_data), 1, 0.5)
adjusted_variable <- "L"
adjusted_model <- "L+I(L^2)"
fit <- CoxTermsearch(sim_data, time_start, time_end, status, exposure, knots = knots, latency,
adjusted_variable = adjusted_variable, adjusted_model = adjusted_model)
#L is selected into the final model but L^2 was not
extract_CoxTermsearch(fit, 10, latency = latency, knot = knots)
```

## resample_clusters *This is a function to help resemple clusters for bootstrap. It is adapted from the function "clusterboot" in the R package "fpc".*

### Description

This is a function to help resemple clusters for bootstrap. It is adapted from the function "clusterboot" in the R package "fpc".

### Usage

```
resample_clusters(data, id = "id")
```

### Arguments

| | |
|---|---|
| data | A data frame containing the data |
| id | A string of the column name that contains the subject ID |

### Value

A data frame with the same structure as the input data, but with resampled clusters

## sim_data *example A simulated data for the use of example*

### Description

example A simulated data for the use of example

### Usage

```
sim_data
```

### Format

An object of class `data.frame` with 9246 rows and 38 columns.

# Index

∗ **datasets**
sim_data, 14

CoxKnotsearch, 2
CoxKnotsearch_boot, 3
CoxNCSpline, 5
CoxPoly, 6
CoxTermsearch, 7
CoxTermsearch_boot, 9

extract_CoxKnotsearch, 10
extract_CoxNCSpline, 11
extract_CoxPoly, 12
extract_CoxTermsearch, 13

resample_clusters, 14

sim_data, 14